```
          "author": "Author",
          "price": 9.90,
          "publisher": "Polarsophia"
        }
        """.formatted(bookIsbn));

    mockWebServer.enqueue(mockResponse);                    ❷

    Mono<Book> book = bookClient.getBookByIsbn(bookIsbn);

    StepVerifier.create(book)                               ❸
      .expectNextMatches(
        b -> b.isbn().equals(bookIsbn))                     ❹
      .verifyComplete();                                    ❺
  }
}
```

❶ Defines the response to be returned by the mock server

❷ Adds a mock response to the queue processed by the mock server

❸ Initializes a StepVerifier object with the object returned by BookClient

❹ Asserts that the Book returned has the ISBN requested

❺ Verifies that the reactive stream completed successfully

Let’s run the tests and ensure they succeed. Open a Terminal window, navigate to the root folder of your Order Service project, and run the following command:

```
$ ./gradlew test --tests BookClientTests
```

NOTE When using mocks, there might be situations where the test results depend on the order in which test cases are executed, which tend to be the same on the same operating system. To prevent unwanted execution dependencies, you can annotate the test class with `@TestMethodOrder(MethodOrderer.Random.class)` to ensure that a pseudo-random order is used at each execution.

After testing the REST client part, you can move on and verify the data persistence layer for Order Service.